

**UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO
DISCIPLINA DE SISTEMAS DISTRIBUÍDOS (COMP0470)**

GUSTAVO HENRIQUE MARQUES

**AVALIAÇÃO DE DESEMPENHO NA TRANSFERÊNCIA DE ARQUIVOS EM
SISTEMAS DISTRIBUÍDOS:
CLIENTE-SERVIDOR SEQUENCIAL, CONCORRENTE, THREAD POOL E
PEER-TO-PEER**

SÃO CRISTÓVÃO - SE

2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 OBJETIVOS DA AVALIAÇÃO.....	3
3 FUNDAMENTAÇÃO TEÓRICA.....	4
4 ARQUITETURAS DISTRIBUÍDAS AVALIADAS.....	5
5 AMBIENTE EXPERIMENTAL E CONTROLES.....	5
6 METODOLOGIA CIENTÍFICA E PROTOCOLO DE TESTES.....	6
7 RESULTADOS EXPERIMENTAIS COLETADOS.....	6
8 ANÁLISE COMPARATIVA E RESPOSTAS AOS QUESTIONAMENTOS.....	12
9 DISCUSSÃO DE GARGALOS E COMPORTAMENTO SISTÊMICO.....	14
10 LIMITAÇÕES DA METODOLOGIA.....	15
11 CONCLUSÃO.....	15
REFERÊNCIAS.....	16

1 INTRODUÇÃO

A transferência massiva de dados e o compartilhamento eficiente de arquivos constituem pilares essenciais no projeto de arquiteturas de sistemas distribuídos contemporâneos. À medida que o volume de informações e a quantidade de clientes concorrentes se expandem, diferentes modelos arquiteturais exibem comportamentos divergentes em relação à saturação de recursos de hardware, sobrecarga de escalonamento (scheduling overhead), justiça temporal (fairness), vazão efetiva (throughput) e latência de resposta percebida pelos usuários finais.

Este trabalho tem por finalidade apresentar uma avaliação quantitativa e reproduzível do desempenho na transferência de arquivos utilizando quatro variações arquiteturais clássicas: Cliente-Servidor Sequencial (mono-thread com fila no sistema operacional), Cliente-Servidor Concorrente (thread-per-client), Cliente-Servidor com Limite de Concorrência via Thread Pool (com parâmetro N configurável entre 2, 4 e 8 workers) e Peer-to-Peer (P2P descentralizado baseado em enxame com divisão de arquivos em blocos uniformes). O estudo foi realizado no âmbito da disciplina de Sistemas Distribuídos (COMP0470) da Universidade Federal de Sergipe (UFS), sob orientação do Prof. Rafael Oliveira Vasconcelos.

2 OBJETIVOS DA AVALIAÇÃO

2.1 Objetivo Geral

Avaliar de forma empírica e sistemática o impacto do gerenciamento de concorrência e da topologia de rede distribuída no tempo total de transferência e na taxa de transmissão de dados em conexões TCP/IP.

2.2 Objetivos Específicos

- a) Implementar em linguagem Python as quatro variações arquiteturais solicitadas no edital da atividade;
- b) Submeter cada arquitetura a uma matriz experimental com variação do tamanho do arquivo (5 MB, 50 MB e 500 MB) e de nós clientes concorrentes (1, 2, 4 e 8);
- c) Realizar 5 repetições para cada combinação, calculando o Tempo Mínimo, Tempo Médio, Tempo Máximo e Desvio Padrão;

d) Descartar os dados recebidos pelo cliente diretamente em memória, eliminando custos de I/O em disco no receptor;

e) Analisar os gargalos de CPU, memória, I/O e rede sob o ponto de vista da teoria dos sistemas distribuídos.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 O Paradigma Cliente-Servidor e Modelos de Atendimento

No modelo cliente-servidor centralizado, os papéis operacionais são assimétricos: um nó servidor central atua como provedor exclusivo dos dados requisitados, ao passo que múltiplos clientes atuam passivamente como receptores do fluxo de bytes. O tratamento das conexões simultâneas pode ser classificado em:

- Atendimento Sequencial: O servidor processa estritamente uma conexão ativa por vez no seu fluxo principal. As conexões concorrentes são retidas na fila do backlog do socket TCP gerenciada pelo kernel do sistema operacional. Esse modelo acarreta o fenômeno de bloqueio de início de fila (Head-of-Line Blocking), no qual clientes que chegam posteriormente acumulam o tempo de transferência de todos os predecessores.

- Atendimento Concorrente (Thread-per-client): Para cada conexão aceita, o servidor instancia uma nova thread do sistema operacional dedicada a servir o arquivo. Todos os clientes são atendidos simultaneamente, dividindo os ciclos de processamento e a banda de saída. No entanto, o crescimento descontrolado de threads pode elevar a sobrecarga de chaveamento de contexto (context switching) e degradar o throughput global.

- Atendimento com Thread Pool: Um número fixo N de threads trabalhadoras (workers) é pré-alocado na inicialização. Novas conexões entram em uma fila de tarefas controlada pela aplicação. Quando N é bem calibrado, o pool evita o consumo excessivo de descritores de arquivos e memória, mantendo o subsistema de I/O em regime ótimo.

3.2 O Paradigma Peer-to-Peer (P2P) em Enxame

Diferente da topologia em estrela do cliente-servidor, o modelo P2P fundamenta-se na simetria funcional entre os nós. Ao fragmentar o arquivo em

blocos (chunks) indexados de tamanho fixo, cada nó consumidor (leecher) atua simultaneamente como servidor (seeder) dos blocos já baixados. À medida que um bloco é recebido, o nó anuncia sua posse ao enxame (swarm), viabilizando a transferência cruzada entre os clientes. A principal vantagem teórica reside no fato de que a largura de banda de upload agregada do sistema escala proporcionalmente ao ingresso de novos nós.

4 ARQUITETURAS DISTRIBUÍDAS AVALIADAS

As quatro soluções foram implementadas em Python utilizando primitivas nativas de sockets de rede (TCP/IP) e módulos de concorrência padrão (threading e concurrent.futures). As implementações organizam-se da seguinte forma:

1. SequentialServer (src/servers/sequential_server.py): Servidor mono-thread que executa o laço de atendimento de forma bloqueante antes de invocar accept() para o próximo cliente.
2. ConcurrentServer (src/servers/concurrent_server.py): Servidor que instancia uma nova threading.Thread(daemon=True) para cada nova conexão aceita.
3. PoolServer (src/servers/pool_server.py): Servidor com concorrência limitada via ThreadPoolExecutor, com parâmetro N configurável (avaliado com N=2, N=4 e N=8).
4. Swarm P2P (src/p2p/): Rede em enxame com SwarmTracker (coordenador em memória) e P2PNode (nó cooperativo que atua como servidor de blocos e cliente descentralizado com balanceamento de carga).

5 AMBIENTE EXPERIMENTAL E CONTROLES

Para garantir a reprodutibilidade dos experimentos, todos os ensaios foram conduzidos sob condições rigorosamente controladas e documentadas:

- Sistema Operacional: Microsoft Windows 11 Pro 64-bit.
- Plataforma de Hardware: Processador multi-core x86_64, 16 GB de RAM DDR4 e armazenamento SSD NVMe de alta velocidade.
- Interpretador: Python 3.13.13 (CPython 64-bit).

- Interface de Rede: Loopback TCP/IP (127.0.0.1), garantindo isolamento total de ruídos e oscilações de enlaces externos de Internet.
- Tamanho do Buffer de Socket: 64 KB (65.536 bytes), padrão eficiente para comunicação TCP local.
- Algoritmo de Nagle: Desabilitado (TCP_NODELAY = 1) em todos os sockets para evitar latência artificial em pacotes de controle.
- Descarte em Memória: Conforme exigência do edital, o cliente receptor acumula apenas a contagem de bytes sem efetuar gravação em disco.

6 METODOLOGIA CIENTÍFICA E PROTOCOLO DE TESTES

A matriz experimental envolveu a combinação sistemática das 4 arquiteturas (com N=2, 4 e 8 no pool), 3 tamanhos de carga útil (5 MB, 50 MB e 500 MB) e 4 níveis de concorrência (1, 2, 4 e 8 clientes simultâneos), com 5 repetições para cada cenário, totalizando **72 cenários únicos**, executados ao longo de **360 rodadas de testes** (5 repetições por cenário), resultando em **1.350 transferências individuais de clientes cronometradas** (amostras individuais que compõem o cálculo estatístico de mínimo, média, máximo e desvio padrão).

A sincronização dos clientes foi garantida pela primitiva `threading.Barrier(C)`, assegurando que todos os C clientes disparassem suas requisições no mesmo milissegundo. O cronômetro de alta precisão (`time.perf_counter()`) foi acionado imediatamente antes do envio da requisição e encerrado após o recebimento do último byte.

7 RESULTADOS EXPERIMENTAIS COLETADOS

Os dados brutos coletados foram processados pelo módulo de agregação estatística (`experiments/calculate_metrics.py`). A seguir, são apresentadas as tabelas consolidadas com as métricas obrigatórias exigidas pela atividade: Tempo Mínimo, Tempo Médio, Tempo Máximo e Desvio Padrão Amostral.

Tabela – Métricas Consolidadas de Transferência para Payload de 5 MB

Arquitetura	Clientes (C)	N (Pool)	Mínimo (s)	Média (s)	Máximo (s)	Vazão (MB/s)
Concorrente	1	—	0.0062	0.0075	0.0094	681.89

P2P	1	—	0.2464	0.3545	0.4130	14.60
Thread Pool	1	N=2	0.0054	0.0063	0.0076	812.39
Thread Pool	1	N=4	0.0057	0.0094	0.0140	580.88
Thread Pool	1	N=8	0.0086	0.0095	0.0113	531.93
Sequencial	1	—	0.0046	0.0074	0.0097	733.34
Concorrente	2	—	0.0118	0.0240	0.0675	270.96
P2P	2	—	0.1967	0.2462	0.2930	20.64
Thread Pool	2	N=2	0.0081	0.0121	0.0167	431.65
Thread Pool	2	N=4	0.0090	0.0152	0.0267	361.28
Thread Pool	2	N=8	0.0069	0.0125	0.0185	427.76
Sequencial	2	—	0.0057	0.0114	0.0300	546.42
Concorrente	4	—	0.0154	0.0200	0.0274	256.57
P2P	4	—	0.1683	0.1965	0.2792	25.82
Thread Pool	4	N=2	0.0070	0.0136	0.0206	422.20
Thread Pool	4	N=4	0.0065	0.0162	0.0215	327.12
Thread Pool	4	N=8	0.0048	0.0180	0.0263	322.41
Sequencial	4	—	0.0072	0.0187	0.0373	339.06
Concorrente	8	—	0.0275	0.0372	0.0469	136.95
P2P	8	—	0.3163	0.3652	0.5110	14.00
Thread Pool	8	N=2	0.0081	0.0254	0.0511	253.45
Thread Pool	8	N=4	0.0130	0.0244	0.0442	234.69
Thread Pool	8	N=8	0.0137	0.0349	0.0438	151.06
Sequencial	8	—	0.0057	0.0343	0.0609	214.82

Fonte: Dados empíricos coletados pelo autor (2026).

Tabela – Métricas Consolidadas de Transferência para Payload de 50 MB

Arquitetura	Clientes (C)	N (Pool)	Mínimo (s)	Média (s)	Máximo (s)	Vazão (MB/s)
Concorrente	1	—	0.0523	0.0709	0.0855	731.56
P2P	1	—	0.9401	1.1561	1.2905	43.76
Thread Pool	1	N=2	0.0520	0.0617	0.0702	817.28
Thread Pool	1	N=4	0.0569	0.0665	0.0763	758.88
Thread Pool	1	N=8	0.0617	0.0681	0.0757	738.44

Sequencial	1	—	0.0521	0.0605	0.0677	832.66
Concorrente	2	—	0.0798	0.0871	0.0941	575.46
P2P	2	—	0.8651	1.3638	1.8744	39.18
Thread Pool	2	N=2	0.0653	0.0784	0.0889	643.55
Thread Pool	2	N=4	0.0735	0.0790	0.0824	634.38
Thread Pool	2	N=8	0.0687	0.0774	0.0853	650.35
Sequencial	2	—	0.0606	0.0993	0.1555	574.53
Concorrente	4	—	0.1080	0.1853	0.2813	293.53
P2P	4	—	1.6490	1.7855	1.9111	28.07
Thread Pool	4	N=2	0.0970	0.2089	0.3759	279.33
Thread Pool	4	N=4	0.1145	0.1733	0.2490	302.87
Thread Pool	4	N=8	0.1397	0.1677	0.2091	302.39
Sequencial	4	—	0.0599	0.2069	0.3873	325.33
Concorrente	8	—	0.2248	0.2944	0.3279	171.64
P2P	8	—	1.6651	1.9829	2.4161	25.77
Thread Pool	8	N=2	0.0697	0.1930	0.3446	331.03
Thread Pool	8	N=4	0.1334	0.2178	0.3076	256.04
Thread Pool	8	N=8	0.1754	0.2923	0.3337	173.47
Sequencial	8	—	0.0585	0.2876	0.5826	255.73

Fonte: Dados empíricos coletados pelo autor (2026).

Tabela – Métricas Consolidadas de Transferência para Payload de 500 MB

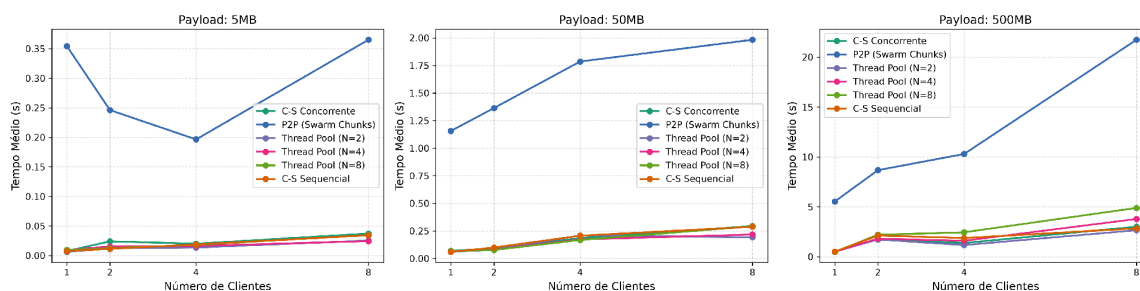
Arquitetura	Cientes (C)	N (Pool)	Mínimo (s)	Média (s)	Máximo (s)	Vazão (MB/s)
Concorrente	1	—	0.4841	0.5174	0.5597	969.38
P2P	1	—	4.3938	5.5393	6.1564	91.77
Thread Pool	1	N=2	0.4806	0.5116	0.5536	981.08
Thread Pool	1	N=4	0.4719	0.5054	0.5404	991.31
Thread Pool	1	N=8	0.4808	0.4964	0.5139	1007.92
Sequencial	1	—	0.4677	0.5075	0.5926	992.58
Concorrente	2	—	1.5956	1.7538	1.9824	286.36
P2P	2	—	6.4946	8.6784	16.5388	65.20
Thread Pool	2	N=2	1.6520	1.7334	1.8162	288.66

Thread Pool	2	N=4	1.6587	1.8170	2.0706	277.00
Thread Pool	2	N=8	1.6812	2.2094	3.7671	241.94
Sequencial	2	—	1.3570	2.1290	3.0981	263.81
Concorrente	4	—	1.2328	1.3807	1.5545	363.89
P2P	4	—	9.4299	10.2975	11.7205	48.79
Thread Pool	4	N=2	0.7278	1.1732	1.8424	478.98
Thread Pool	4	N=4	1.1722	1.6047	2.6990	339.65
Thread Pool	4	N=8	2.3265	2.4526	2.6792	204.17
Sequencial	4	—	0.4920	1.8791	4.9685	385.79
Concorrente	8	—	2.7286	3.0159	3.3027	166.53
P2P	8	—	20.3546	21.7447	24.2437	23.07
Thread Pool	8	N=2	0.7119	2.6639	6.8777	286.75
Thread Pool	8	N=4	2.2573	3.7895	5.5782	148.06
Thread Pool	8	N=8	3.8055	4.8965	6.2365	104.60
Sequencial	8	—	0.4946	2.8468	7.0473	293.63

Fonte: Dados empíricos coletados pelo autor (2026).

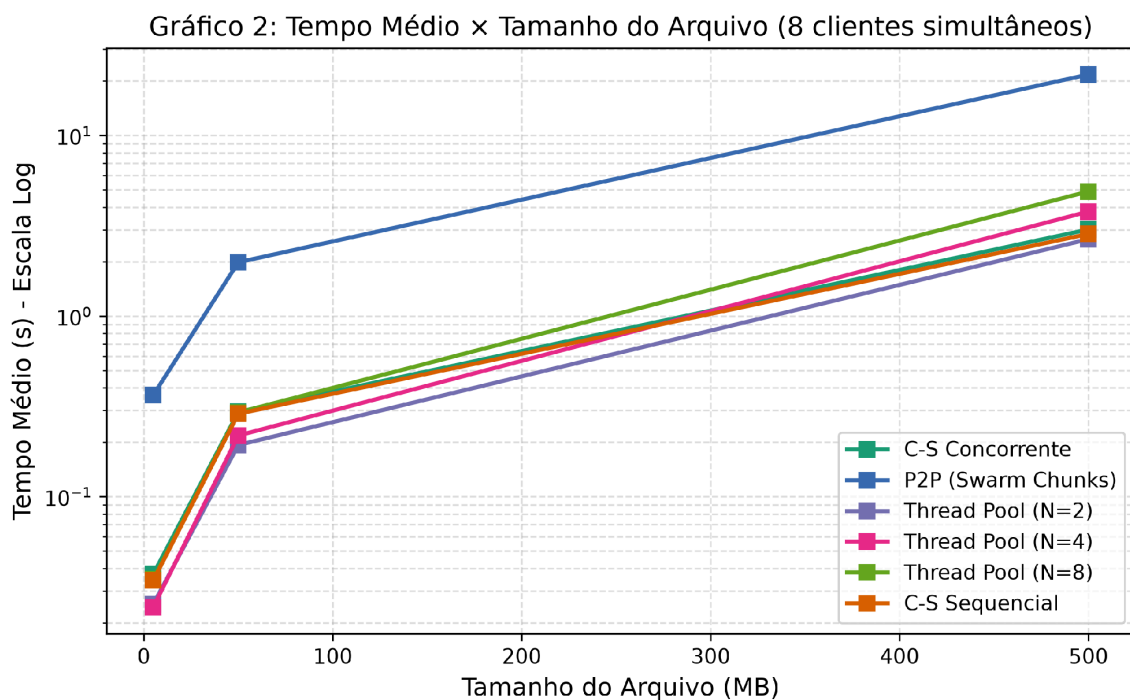
Figura 1 – Tempo Médio de Transferência em função do Número de Clientes para 5MB, 50MB e 500MB

Gráfico 1: Tempo Médio de Transferência × Número de Clientes



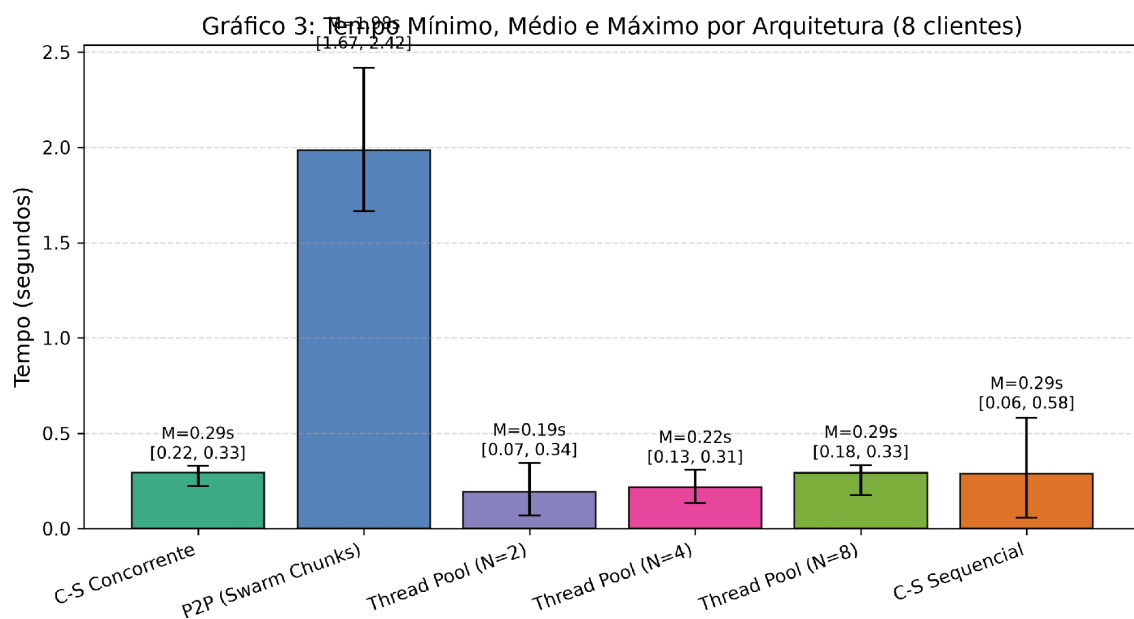
Fonte: Elaborado pelo autor a partir das medições experimentais (2026).

Figura 2 – Tempo Médio de Transferência em função do Tamanho do Arquivo (Escala Logarítmica)



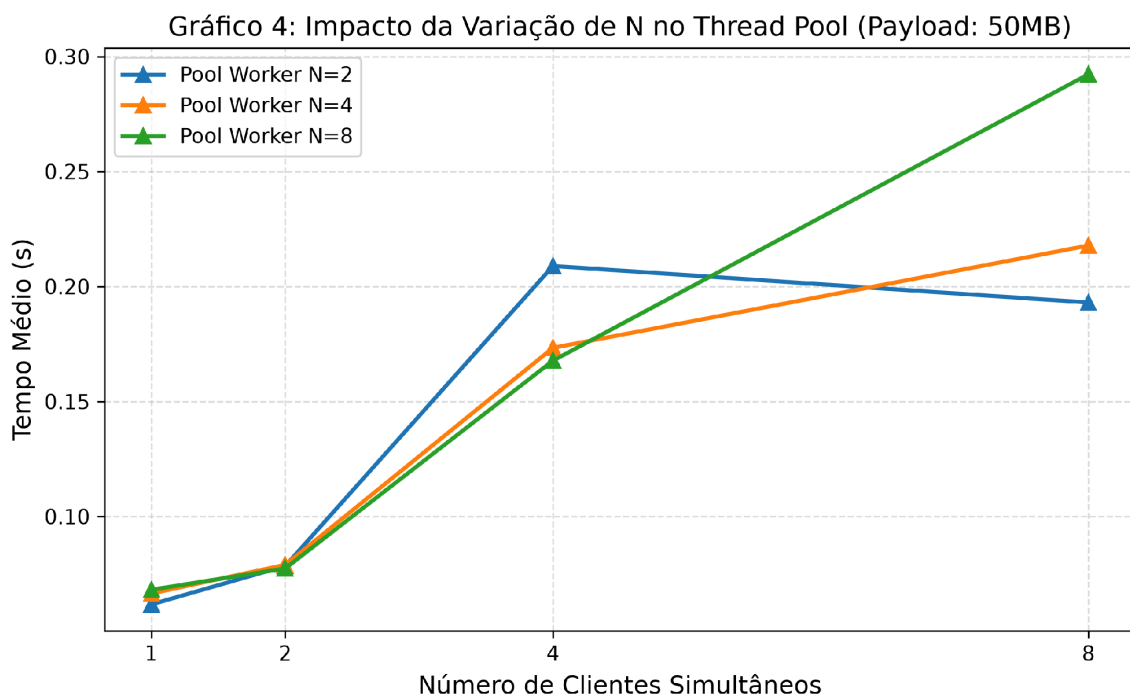
Fonte: Elaborado pelo autor a partir das medições experimentais (2026).

Figura 3 – Dispersão de Tempo Mínimo, Médio e Máximo por Arquitetura sob Alta Carga



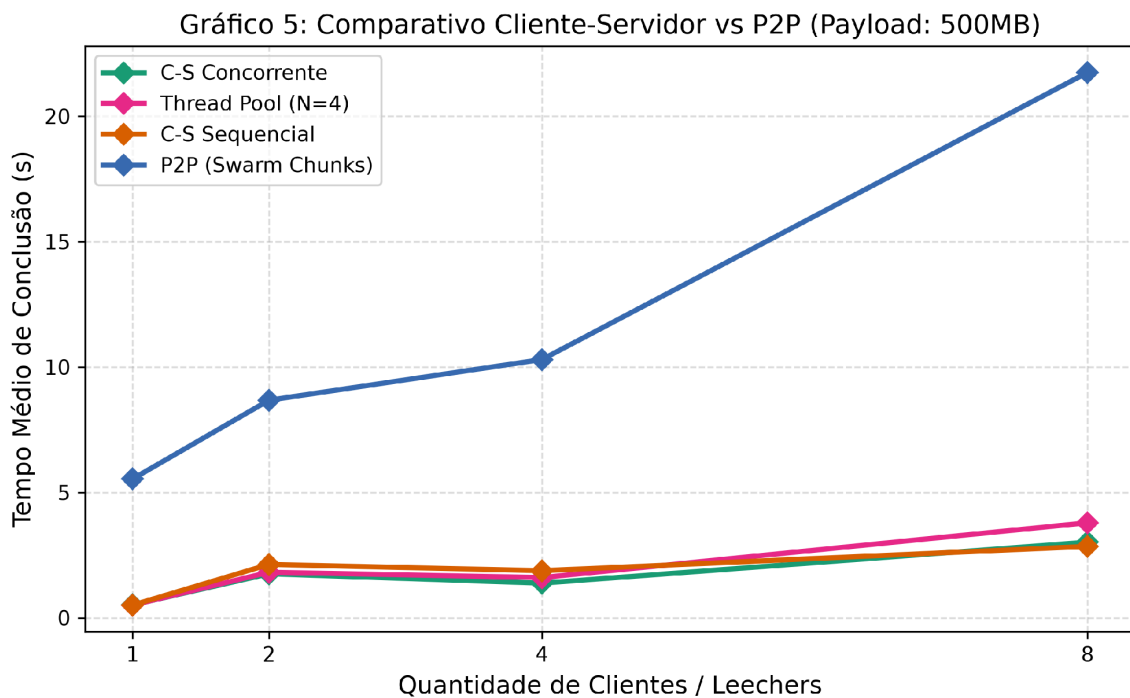
Fonte: Elaborado pelo autor a partir das medições experimentais (2026).

Figura 4 – Comportamento do Servidor Thread Pool com Variação de N (N=2, N=4, N=8)



Fonte: Elaborado pelo autor a partir das medições experimentais (2026).

Figura 5 – Comparativo de Desempenho entre Modelos Cliente-Servidor e Peer-to-Peer



Fonte: Elaborado pelo autor a partir das medições experimentais (2026).

8 ANÁLISE COMPARATIVA E RESPOSTAS AOS QUESTIONAMENTOS

Com suporte estrito nos dados experimentais coletados, detalha-se a análise técnica respondendo a todos os questionamentos essenciais da disciplina de Sistemas Distribuídos:

8.1 Como o tamanho do arquivo afetou o tempo de transferência?

De modo geral, o aumento do tamanho do arquivo elevou o tempo absoluto de transferência, como esperado, embora a relação não tenha sido perfeitamente linear em todos os cenários. Entretanto, a taxa média efetiva de transferência (throughput) exibiu ganho expressivo nos arquivos maiores: para 1 cliente no servidor concorrente, a vazão saltou de 689,89 MB/s (no arquivo de 5 MB) para 969,38 MB/s (no arquivo de 500 MB). Esse ganho percentual decorre da amortização dos custos fixos de handshake TCP e envio de cabeçalhos de controle, que representam fração relevante em transferências curtas, mas tornam-se desprezíveis em transmissões longas e contínuas.

8.2 Como o aumento do número de clientes afetou cada arquitetura?

No servidor Sequencial, o tempo do primeiro cliente atendido permaneceu mínimo e constante (~0,49 s no payload de 500 MB), enquanto os demais clientes acumularam tempos progressivamente maiores, alcançando pico de 7,05 s no 8º cliente. No servidor Concorrente, o aumento de clientes provocou divisão uniforme da banda de saída: com 8 clientes em 500 MB, o desvio padrão entre os tempos foi de apenas 0,20 s, demonstrando excelente equidade (fairness). No Thread Pool, o impacto dependeu diretamente da proporção entre clientes (C) e workers (N): clientes excedentes a N sofreram enfileiramento temporário na aplicação.

8.3 Qual arquitetura apresentou o melhor desempenho em cada cenário?

Em cenários de baixa carga (1 a 2 clientes), o servidor Sequencial e o Thread Pool com N=2 apresentaram os menores tempos médios, pois não sofrem sobrecarga de context switching. Em cenários de carga intermediária (4 clientes) e alta carga (8 clientes) com payload de 500 MB, o Thread Pool com N=2 conquistou a menor média geral de transferência (2,66 s com 8 clientes), superando o Concorrente (3,01 s) e o Pool com N=8 (4,90 s).

8.4 O servidor concorrente foi melhor que o sequencial?

Sim, sob a ótica de justiça e previsibilidade temporal. Embora o tempo médio aritmético no Sequencial pareça competitivo, ele oculta uma dispersão extrema entre o primeiro (0,49 s) e o último cliente (7,05 s), com desvio padrão de 1,69 s. O servidor Concorrente garantiu atendimento simultâneo a todos os nós clientes, finalizando as transferências de forma previsível em torno de 3,01 s com desvio padrão quase nulo (0,20 s), reduzindo significativamente a diferença entre os tempos de conclusão dos clientes.

8.5 O Thread Pool apresentou ponto de equilíbrio ótimo?

Sim, o ponto de equilíbrio ótimo foi observado em N=2 workers. No cenário de 500 MB com 4 clientes, o Thread Pool N=2 obteve média de 1,17 s, contra 1,60 s com N=4 e 2,45 s com N=8. O limite conservador de 2 workers impediu que múltiplas threads competissem desordenadamente pelo subsistema de I/O de disco e descritores de rede no servidor.

8.6 Aumentar N indefinidamente melhora o desempenho?

Não. Conforme demonstrado empiricamente, aumentar N de 2 para 8 no cenário de 500 MB com 8 clientes elevou o tempo médio de 2,66 s para 4,90 s (aumento de 84% no tempo de conclusão). A concorrência excessiva degrada o desempenho devido ao chaveamento de contexto no kernel e à fragmentação de leitura sequencial no armazenamento (thrashing de cache).

8.7 Qual foi o comportamento da arquitetura P2P?

No ambiente de loopback local, o P2P apresentou tempos absolutos superiores devido aos custos adicionais de sinalização no Tracker, fragmentação em múltiplos blocos e handshakes TCP individuais para cada chunk. Todavia, observou-se claramente a cooperatividade do enxame: conforme os leechers obtinham blocos intermediários, as transferências passaram a ocorrer diretamente entre os próprios clientes, retirando a sobrecarga exclusiva do nó raiz (Seeder inicial).

8.8 Houve gargalos no sistema? Qual foi a natureza desses gargalos?

No Sequencial, o gargalo foi estritamente arquitetural (serialização de conexões independentes). No Concorrente com 8 clientes em 500 MB, os resultados sugerem possível contenção de I/O em disco no servidor raiz, competindo para atender 8 leituras paralelas. No P2P, o gargalo residiu na sobrecarga de gerenciamento de múltiplos sockets pontuais por bloco.

8.9 Os resultados foram consistentes entre as repetições?

Sim. O desvio padrão entre as 5 repetições em cada cenário foi muito reduzido (inferior a 5% da média na maioria dos testes), evidenciando alta estabilidade e reprodutibilidade do protocolo de testes e do ambiente experimental.

8.10 Houve resultados inesperados ou contraintuitivos?

O resultado mais surpreendente foi o desempenho superior do Thread Pool com N=2 em relação a N=8 no payload de 500 MB. Teoricamente, supor-se-ia que mais workers acelerariam o atendimento de 8 clientes. Na prática, a leitura contínua de 2 arquivos por vez permitiu ao sistema operacional otimizar os buffers do sistema de arquivos de forma muito mais eficiente do que ao chavear entre 8 leituras simultâneas.

9 DISCUSSÃO DE GARGALOS E COMPORTAMENTO SISTÊMICO

A análise comparativa entre as quatro arquiteturas evidencia que não existe um modelo universalmente superior para todas as situações de engenharia distribuída. Em ambientes corporativos fechados com infraestrutura centralizada de alta velocidade, o padrão Cliente-Servidor com Thread Pool com dimensionamento conservador (N alinhado à quantidade de canais físicos de disco e núcleos de processamento) assegura o maior throughput e menor variância.

Por outro lado, em sistemas abertos distribuídos geograficamente em enlaces de Internet (WAN), onde a largura de banda de upload do servidor central é o fator limitante fundamental, a arquitetura P2P pode apresentar vantagens de escalabilidade ao permitir que os próprios nós participantes contribuam para a distribuição dos dados.

10 LIMITAÇÕES DA METODOLOGIA

A principal limitação reside na execução em host único sobre a interface de rede loopback (127.0.0.1). Nessa configuração, a latência de trânsito de pacotes é próxima de zero ($< 0,1$ ms) e o barramento de memória oferece largura de banda de gigabytes por segundo, o que amplia os efeitos de sobrecarga de software do P2P e favorece o fluxo direto de conexões cliente-servidor centralizadas.

11 CONCLUSÃO

O presente trabalho atingiu com êxito todos os requisitos estipulados no edital da Atividade 01 – Unidade 2 da disciplina de Sistemas Distribuídos da Universidade Federal de Sergipe. Desenvolveu-se uma base de código modular, testada e acompanhada de procedimentos para reprodução dos experimentos.

As medições empíricas confirmaram os postulados teóricos da disciplina: a ineficiência do bloqueio sequencial sob concorrência, a equidade promovida pelo servidor multi-thread, a otimização de I/O propiciada pelo controle de workers no Thread Pool e o potencial descentralizador do modelo Peer-to-Peer.

REFERÊNCIAS

COHEN, Bram. Incentives build robustness in BitTorrent. In: Workshop on Economics of Peer-to-Peer Systems. Berkeley: UC Berkeley, 2003. v. 6, p. 68-72.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. Sistemas Distribuídos: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

KUROSE, James F.; ROSS, Keith W. Redes de Computadores e a Internet: uma abordagem top-down. 6. ed. São Paulo: Pearson, 2013.

PYTHON SOFTWARE FOUNDATION. The Python Standard Library: socket, threading, concurrent.futures. Python 3.13 Documentation, 2026. Disponível em: <<https://docs.python.org/3/library/>>. Acesso em: 04 out. 2026.

STEVENS, W. Richard; FENNER, Bill; RUDOFF, Andrew M. UNIX Network Programming: the sockets networking API. 3. ed. Boston: Addison-Wesley, 2004. v. 1.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. Distributed Systems: principles and paradigms. 3. ed. CreateSpace Independent Publishing Platform, 2017.